

Carte de remplacement pour minuterie — Documentation complète

Traduction. Ceci est la traduction française du manuel. La version faisant foi est l'original allemand ([Deutsch](#)). Également disponible : [English](#).

Projet : Remplacement d'une carte de commande de minuterie défectueuse/obsolète par un développement maison à base d'ESP32-C3, écran OLED, configuration Wi-Fi et Shelly 1PM Mini Gen4 comme interrupteur de puissance.

État : 2026-07-13 · Version de modèle v8 · Phase : vérification avant layout **Auteur :** Silvio (matériel/mesures) + Claude (analyse/conception)

1. Ce que fait l'appareil (description fonctionnelle)

L'appareil possède trois boutons-poussoirs en façade. Une pression sur un bouton active une charge 230 V pendant une durée réglable :

Bouton	Durée par défaut	réglable
T1	5 secondes	1-600 s via Wi-Fi
T2	10 secondes	1-600 s via Wi-Fi
T3	15 secondes	1-600 s via Wi-Fi

Un petit écran OLED derrière la fenêtre de visualisation affiche en haut la **réception Wi-Fi** (à gauche), la grande **heure** (NTP, au centre) et l'état **Repos** (à droite) ; en bas à gauche la **date** (p. ex. « Do 16.07.2026 ») et à droite la **mémoire libre**. Lorsqu'un minuteur est en cours, le **compte à rebours en secondes** s'affiche au centre et **Repas** à droite. Les trois boutons (**S1 Down/Manual**, **S2 SET**, **S3 UP**) déclenchent des minuteurs, les arrêtent et permettent de parcourir un menu d'information — détails au chap. 6.5.

L'utilisation se fait **sans application et sans cloud** via une petite page web optimisée pour les appareils mobiles, que l'ESP32 héberge lui-même sur le **port 80** (accessible dans le navigateur à <http://feeder-relais.local>) : déclencher des boutons, régler les durées, consulter les valeurs réseau et d'état. Pour l'intégration (p. ex. ioBroker), l'ESP propose en plus une **API JSON** légère (voir chap. 6.4). En option, l'appareil peut aussi être intégré à Home Assistant ou ioBroker (adaptateur esphome).

1.1 La chaîne de signal (le trajet du signal)

```
Taster —> ESP32-C3 —> Vorwiderstand 330 Ω —> PhotoMOS-Relais —> Shelly
(T1-T3)   (zählt die   (begrenzt den   (schaltet 230-V-   SW-Eingang
           Zeit, steuert LED-Strom im   Signal galvanisch
           das OLED)   PhotoMOS)   getrennt)
                                     |
                                     ▼
                               Shelly-Relais
                               schaltet Last (0)
```

Pourquoi un Shelly ? Le Shelly 1PM Mini Gen4 assure la commutation proprement dite de la charge, mesure la puissance au passage et est en outre accessible par Wi-Fi/application. Notre carte

ne fait qu'« appuyer » sur l'interrupteur du Shelly — sous forme d'un vrai signal matériel, **sans dépendance au Wi-Fi** pour la fonction principale.

Qu'est-ce qu'un PhotoMOS ? Un minuscule relais à semi-conducteurs : d'un côté une LED (que l'ESP pilote avec 3,3 V), de l'autre un interrupteur commandé par la lumière, autorisé à commuter la tension secteur. Entre les deux il n'y a que de la lumière — donc une isolation galvanique complète entre la très basse tension (TBTS) et le 230 V. L'entrée SW du Shelly est référencée à la tension secteur (elle attend un L commuté), c'est pourquoi il faut **impérativement un modèle à ≥ 400 V de tension de blocage** (p. ex. Omron G3VM-601BY ou Panasonic AQY216). Un optocoupleur ordinaire (PC817) n'est **pas** admissible ici.

Le Shelly est configuré en mode d'entrée « **Interrupteur/Follow** » : son relais reste activé exactement aussi longtemps que l'ESP maintient le signal. Le timing est entièrement géré par l'ESP.

2. Sécurité — à lire en premier

- L'appareil fonctionne avec la **tension secteur 230 V. Danger de mort !** Le montage, les essais et la mise en service doivent être confiés à une personne possédant la qualification appropriée (électricien qualifié ou personne avertie).
- Les premiers tests fonctionnels de l'ESP, des boutons et de l'OLED se font **exclusivement via USB** — sans aucune tension secteur raccordée (voir le plan de test dans le plan de projet).
- Le premier essai sous 230 V se fait derrière un **DDR/PRCD (prise de protection des personnes)**, boîtier fermé ; ne jamais manipuler sous tension dans un appareil ouvert.
- Sur la carte, la règle est la suivante : entre tous les réseaux 230 V et la très basse tension, **au moins 6 mm de distance** (ligne de fuite), détails au chap. 7.6.
- Un fusible (1 A temporisé) et une varistance en amont du module d'alimentation sont obligatoires — les modules (TSP-05/HLK) ne les intègrent pas eux-mêmes.

3. Nomenclature (BOM)

Réf	Composant	Valeur / Type	Remarque
U1	ESP32-C3 Super Mini	18 × 24 mm, USB-C	Contrôleur, Wi-Fi
U2	Relais PhotoMOS	Omron G3VM-601BY ou Panasonic AQY216	≥ 400 V ! SOP-4/DIP-4
PS1	Module AC/DC	TENSTAR TSP-05 (5 V / 3 W)	clone HLK-PM05, 34,7 × 20,5 × 15,05 mm, mesurer les entraxes de broches !
—	Shelly 1PM Mini Gen4	S4SW-001P8EU	29 × 34 × 16 mm, max. 8 A/240 V
—	Écran OLED	0,91" SSD1306, 128 × 32, I2C	Module 38 × 12 mm, broches : GND VCC SCL SDA, adresse 0x3C
SW1-3	Bouton-poussoir à course courte	6 × 6 mm, course 1,5 mm	empreinte broches 6,9 × 4,4 mm (relevée sur l'original)
R1	Résistance	330 Ω , axiale	résistance série LED du PhotoMOS
F1	Fusible fin	1 A temporisé, 5 × 20 mm + porte-fusible	côté primaire
RV1	Varistance	S14K275	côté primaire en parallèle
C1	Condensateur électrolytique	220 μ F / 10 V	tampon 5 V pour les pics de courant Wi-Fi

Réf	Composant	Valeur / Type	Remarque
C2	Condensateur céramique	100 nF	découplage 5 V
J1	Bornier secteur	2 pôles (L, N)	bornier à vis ou pastilles à souder
J2	Raccordement Shelly	4 pastilles/broches à souder	L, N, SW, O — courts fils vers les borniers à vis du Shelly
J3	Sortie de charge	2 pôles	O (commuté), N
J4	Embase OLED	barrette femelle 1 × 4, pas 2,54	hauteur de montage selon la mesure du stack (~11 mm), chap. 5.6
—	Barrette mâle 1 × 4	pas 2,54	soudée à l'OLED (vers l'arrière)
—	Matériel de fixation	ruban 3M-VHB ou clip 3D	fixation Shelly + TSP

4. Boîtier et mécanique

4.1 Le système de coordonnées (important pour toute la suite !)

Toutes les cotes se réfèrent au **coin supérieur gauche de la CARTE ORIGINALE**, x vers la droite, **y vers le bas**, vue sur le **côté composants** (boutons en haut). La nouvelle carte, plus grande, déborde de cette origine — d'où les **coordonnées négatives**. Avantage : toutes les valeurs déjà mesurées restent valables pour toujours, quelle que soit la façon dont la découpe s'agrandit.

4.2 Conditions aux limites du boîtier (mesurées/déduites)

Grandeur	Valeur	Source
Hauteur intérieure totale	19,9 mm	Mesure Silvio
Hauteur du cadre (face avant)	2 mm	Mesure Silvio
Espace au-dessus de la carte	16,5 mm	Mesure Silvio (carte ↔ face avant)
Épaisseur de la carte	1,6 mm	Standard
Espace sous la carte	≈ 1,8 mm	19,9 – 16,5 – 1,6 → rien ne doit dépasser vers le bas !
Limite de hauteur dans la zone de la fenêtre	≈ 12 mm	verre + OLED enfiché devant
Fixation de la carte	4 vis sur bossages de 16,5 mm à la face avant	avec nervures de stabilisation vers le bord
Assemblage du boîtier	6 vis extérieures	la carte a 6 encoches en U ouvertes sur le bord (largeur 10) au lieu de perçages fermés

4.3 Géométrie finale de la carte (modèle v9)

Toutes les valeurs figurent **en format lisible par machine** dans `hardware/platine_template.py` (la seule source de vérité !). Résumé :

Élément	Coordonnées (mm)	Précision
Découpe	x –14,2...91,1 / y –11,8...67,6 → 105,3 × 79,4 , rayon d'angle 4 (localement réduit à 3,2 resp. 2,1 aux encoches de bossage extérieures)	±1 (à gauche ±1,5)

Élément	Coordonnées (mm)	Précision
4 trous de fixation $\varnothing 3,0$	(3,8/4,1) (73,1/4,0) (3,7/39,6) (73,4/50,7)	$\pm 0,3$
6 encoches de bossage (U ouvert sur le bord, largeur 10)	rangée du haut y $-6,0$ (ouvre vers le bord supérieur), rangée du bas y $+64,0$ (ouvre vers le bord inférieur), entraxe des rangées 70 ; colonnes x $-6,0$ / 39,0 / 84,0 (pas 45)	mesuré directement (Silvio 13.07.) ; position absolue encore à essayer
Boutons T1/T2/T3 (centres)	(13,2/12,3) (38,8/12,2) (64,1/12,0)	$\pm 0,3$
Fenêtre de visualisation (référence)	x 18,4...53,6 / y 23,3...40,0	± 1
J4 embase OLED (4 broches vert.)	broches à x = 15,0, y = 27,84 / 30,38 / 32,92 / 35,46	vérifier les hypothèses, chap. 5.6
Zone d'implantation Shelly	x 56...90 / y 16,5...45,5 (couché)	—
Zone d'implantation ESP32-C3	x 8,5...32,5 / y $-10...8$ (en travers, USB vers le haut)	—
Zone d'implantation TSP-05	x $-1,5...33,2$ / y 44,2...64,7 (côté AC vers la gauche)	—
Keepouts (référence)	$\varnothing 8$ autour des 4 têtes de vis + 5 bandes de nervures	directions des nervures = hypothèse !

D'où viennent les valeurs ? Scan côté soudure de la carte originale (300 dpi, redressé, miroité) pour les boutons/trous/découpe ; scan du boîtier, recalé sur les 4 vis de cadre (transformation miroir $x_{\text{platine}} = 110,5 - x_{\text{gehäuse}}$, $y_{\text{platine}} = y_{\text{gehäuse}} - 16,4$, $\pm 0,4$ mm) ; position de la fenêtre à partir de la photo redressée en perspective.

4.4 L'ancienne découpe

La découpe trapézoïdale au bord inférieur de la carte originale était le dégagement du bossage de vissage central inférieur (B5). Dans la nouvelle carte elle est **remplacée par l'encoche en U ouverte sur le bord B5** et disparaît.

5. Électrique en détail

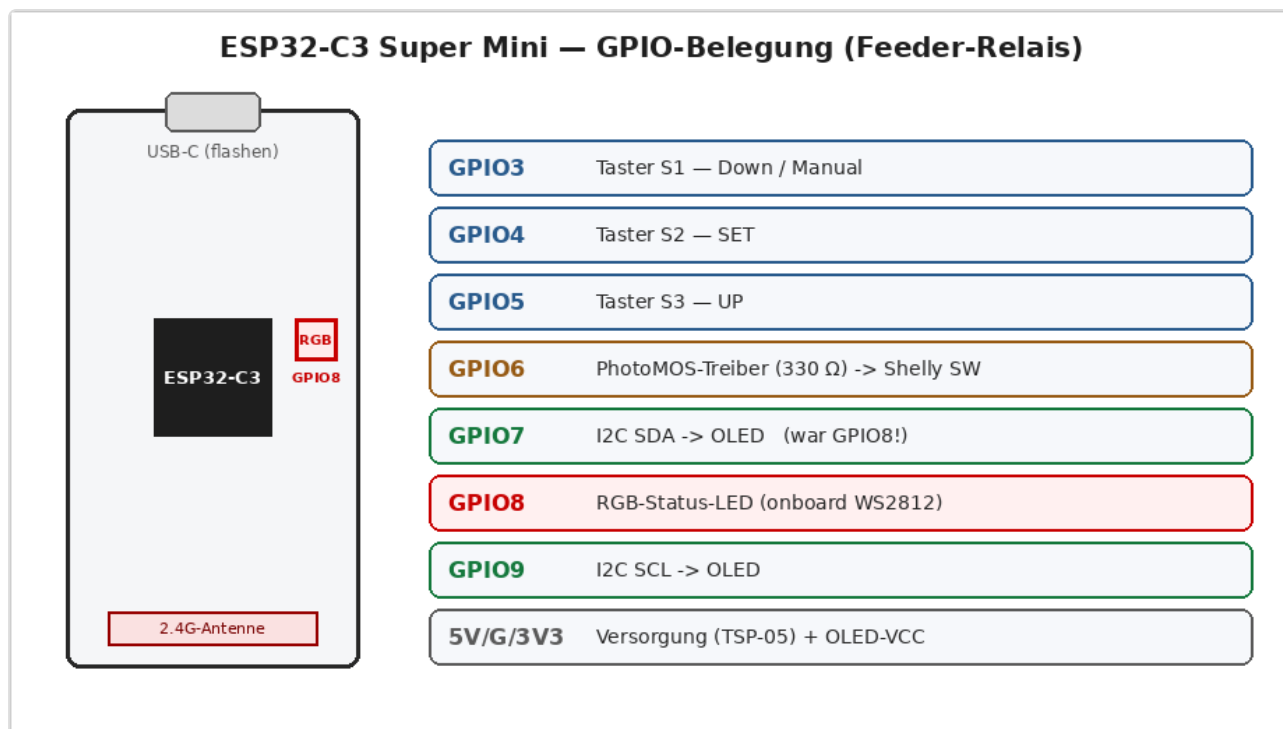
5.1 Vue d'ensemble des réseaux (connexions)

Nom du réseau	conduit	relie
L_IN	230 V	bornier secteur J1.L → fusible F1
L_F	230 V (protégé)	F1 → varistance, TSP-05 AC, PhotoMOS OUT1, Shelly L
N	230 V neutre	J1.N → varistance, TSP-05 AC, Shelly N, charge J3
SW_SHELLY	230 V commuté	PhotoMOS OUT2 → Shelly SW
O_LAST	230 V commuté	Shelly O → sortie de charge J3
+5V	très basse tension	TSP-05 +Vo → ESP 5V, C1, C2
GND	très basse tension	TSP-05 -Vo → ESP GND, boutons, PhotoMOS K, OLED
+3V3	très basse tension	sortie 3V3 de l'ESP → OLED VCC
BTN1/2/3	signal	boutons → GPIO3/4/5 (pull-ups internes)
PMOS_DRV → PMOS_A	signal	GPIO6 → R1 330 Ω → anode PhotoMOS

Nom du réseau	conduit	relie
SDA / SCL	I2C	GPIO7 / GPIO9 → OLED
RGB-LED	signal	GPIO8 → WS2812 onboard (LED d'état, atténuée)

5.2 ESP32-C3 Super Mini : carte et brochage

Variante de module « ESP32-C3FN4 ». Aperçu des broches que nous utilisons :



Composants sur le module : **1** USB-Type-C · **2** bouton BOOT · **3** bouton RESET · **4** LDO CAT6219 (3,3 V, 500 mA) · **5** ESP32-C3FN4 · **6** antenne 2,4 GHz · **7** module RGB (**WS2812**, **GPIO8**) · **8** connecteur d'antenne (u.FL).

Broche	Fonction	Remarque
5V / G	Alimentation	depuis le TSP-05
3V3	Sortie du LDO onboard	alimente l'OLED
GPIO3	Bouton T1	pull-up interne, bouton vers GND
GPIO4	Bouton T2	idem
GPIO5	Bouton T3	idem
GPIO6	Pilote PhotoMOS	high = Shelly activé
GPIO7	SDA (I2C)	OLED — au lieu de GPIO8 , où se trouve la LED RGB
GPIO8	LED RGB (WS2812)	LED d'état onboard, atténuée
GPIO9	SCL (I2C)	OLED

Volontairement évités : GPIO2/8/9 comme boutons (broches de strapping). **GPIO8 porte la LED RGB onboard (WS2812)** — c'est pourquoi I2C-SDA est sur **GPIO7** (pas GPIO8), sinon le trafic I2C piloterait la LED en pleine luminosité (chaleur). SCL reste sur GPIO9 (compatible boot avec un pull-up I2C).

5.3 Shelly 1PM Mini Gen4

- Bornes : **SW** (entrée de commutation), **O** (sortie de charge), **L**, **N**.
- Alimentation 110–240 V~, relais max. 8 A / 2000 W @ 240 V.
- Configuration après mise en service : Input Mode = Switch (Follow), pour que le relais suive le signal SW au 1:1.
- Montage : couché sur la zone d'implantation, ruban VHB ou clip 3D. Quatre courts fils (p. ex. 0,75 mm², 6–7 mm dénudés) des borniers à vis vers les pastilles J2.

5.4 Alimentation TSP-05

100–240 V~ → 5 V DC / 3 W (= 600 mA). Suffisant pour les pics Wi-Fi de l'ESP (~350 mA) plus l'OLED avec de la marge. **À vérifier : contrôler les entraxes de broches au pied à coulisse par rapport à l'empreinte KiCad Converter_ACDC_HiLink_HLK-PMxx** (le clone est généralement compatible broche à broche — mais avec du 230 V on ne suppose rien).

5.5 Boutons

Les trois boutons à course courte 6×6 (course 1,5 mm) sont actionnés par des poussoirs dans la face avant — leur position n'est donc **pas négociable** (implanter à ±0,2 mm). Empreinte de broches par bouton : 6,9 mm × 4,4 mm.

5.6 OLED — montage enfiché (« stack »)

L'OLED n'est **pas soudé**, mais **enfiché sur J4** : barrette femelle 1×4 sur la carte, barrette mâle à l'OLED vers l'arrière, la face de l'écran s'appuie (avec un fin patin en mousse) contre la vitre.

- Affectation de J4 de haut en bas : **GND, VCC (3V3), SCL, SDA** — identique au marquage du module existant.
- **Hauteur de stack requise** = 16,5 mm – enfoncement du verre – épaisseur du module (≈ 2,8 mm) ≈ **13–14 mm**. Réalisation p. ex. barrette femelle 11 mm + collerette de barrette mâle 2,5 mm. Les barrettes femelles existent en 8,5/11/13/15 mm.
- **Mesurer sur le module réel avant de figer** (les fabricants varient de ±1 mm) : 1. distance bord gauche du module → rangée de broches (hypothèse 2,5 mm), 2. distance bord gauche du module → centre de la surface active (hypothèse 23,5 mm), 3. enfoncement du verre (face intérieure avant → dessous du verre). Concrètement : poser le module dans le cadre, écran centré dans la fenêtre, mesurer les trous de broches par rapport à deux vis de bossage → c'est J4.

6. Firmware (ESPHome)

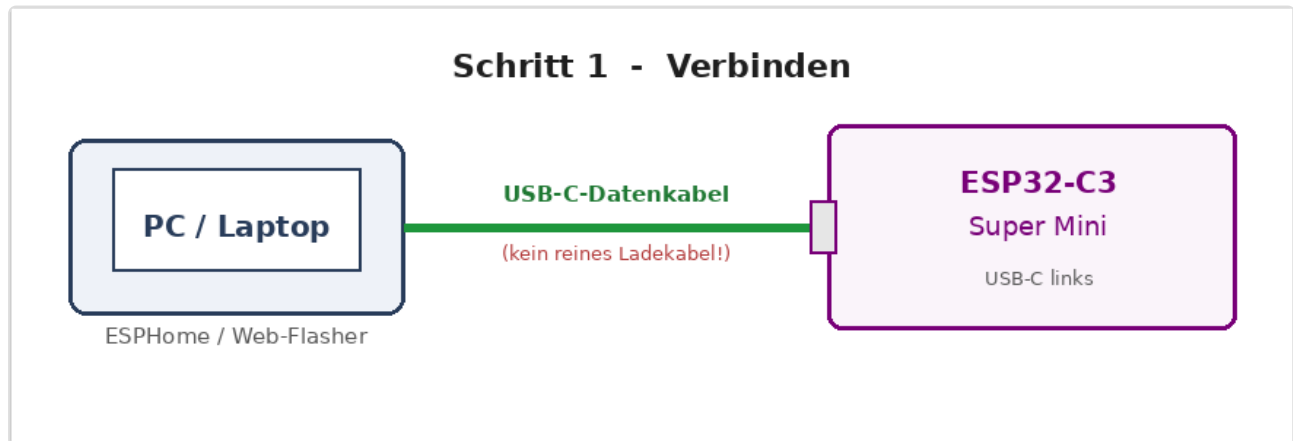
Fichier : `firmware/timer-relais-c3.yaml` (nom d'appareil **feeder-relais**, mDNS `feeder-relais.local`). Contient tout : Wi-Fi avec hotspot de repli (« Feeder-Relais Setup », portail captif), **horloge NTP** (`de.pool.ntp.org`), web-app mobile + API JSON (chap. 6.4), les trois durées persistantes, la logique des boutons avec anti-rebond/appui long/menu (chap. 6.5) et un OLED avec état Wi-Fi, heure resp. compte à rebours.

6.1 Premier flash — étape par étape (pour débutants)

La **toute première fois**, le firmware arrive sur l'ESP32-C3 **par câble USB**. Toutes les mises à jour ultérieures se font ensuite sans fil (chap. 6.6).

Ce dont tu as besoin :

- l'**ESP32-C3 Super Mini**,
- un **câble de données USB-C** — attention : beaucoup de câbles bon marché ne savent que charger ; le chip **n'est alors pas reconnu !**
- un PC avec **Google Chrome** ou **Microsoft Edge** (voie B, navigateur) — ou ESPHome sur le PC (voie A).



Des images prêtes à l'emploi sont disponibles — tu n'as donc pas à compiler toi-même :

- dans le dépôt sous **firmware/build/** (toujours l'état actuel) :
- `feeder-relais.factory.bin` — image complète pour le **premier flash** (adresse 0x0)
- `feeder-relais.ota.bin` — image applicative pour **OTA/mise à jour web** (chap. 6.6)
- comme **release GitHub** (nom de fichier versionné, p. ex. `feeder-relais-v0.0.1.factory.bin`) dans la section Releases du dépôt.

Reconstruire soi-même : `./firmware/build_images.sh` (compile et met à jour `firmware/build/`). Les fichiers bruts se trouvent sinon sous `firmware/.esphome/build/feeder-relais/.pioenvs/feeder-relais/`.

Les images ne contiennent AUCUN identifiant Wi-Fi. La configuration se fait après le flash via le hotspot de configuration (chap. 6.2). Ainsi aucun réseau Wi-Fi étranger n'entre dans la livraison, et les données configurées survivent aux mises à jour OTA (clé de stockage fixe).

Voie A — avec ESPHome (recommandée, une commande)

Un `secrets.yaml` n'est **pas** nécessaire — aucune donnée Wi-Fi n'est compilée.

1. Brancher l'ESP en **USB-C**.
2. Exécuter dans le dossier du projet : `esphome run firmware/timer-relais-c3.yaml` ESPHome compile, demande le **port série** (sous Linux p. ex. `/dev/ttyACM0`, sous Windows un `COMx`) et flashe.
3. Après le redémarrage, l'ESP ouvre le hotspot de configuration → poursuivre au chap. 6.2.

Astuce Linux : en cas de « Permission denied » sur `/dev/ttyACM0`, ajouter ton utilisateur au groupe `dialout` : `sudo usermod -aG dialout $USER` (se reconnecter).

Voie B — dans le navigateur, sans installation (esptool-js)

Pour le `firmware.factory.bin` prêt à l'emploi, entièrement sans ESPHome :

Ablauf - Erstes Flashen ueber den Browser (esptool-js)

- 1 **Chrome oder Edge oeffnen**
espressif.github.io/esptool-js
- 2 **ESP per USB-C anstecken**
USB-C-Datenkabel verwenden
- 3 **"Connect" -> Port waehlen**
ESP32-C3 als serieller Port
- 4 **Flash Address 0 + factory.bin**
Datei waehlen, dann "Program"
- 5 **WLAN einrichten**
AP "Feeder-Relais Setup" -> feeder-relais.local

1. Dans **Chrome/Edge**, ouvrir la page <https://espressif.github.io/esptool-js/>.
2. Brancher l'ESP en **câble de données USB-C**.
3. Laisser le débit 115200, cliquer **Connect** et choisir dans la fenêtre le port série de l'ESP (souvent appelé « USB JTAG/serial debug unit » ou un COMx).
4. Dans **Flash Address**, saisir 0, avec **Choose File** sélectionner le `firmware.factory.bin`, puis **Program**. Attendre que « Hard resetting... » apparaisse.
5. Poursuivre avec la configuration Wi-Fi (chap. 6.2).

Si « Connect » ne fonctionne pas : forcer le **mode boot** — maintenir **BOOT** appuyé, appuyer brièvement sur **RESET**, relâcher **BOOT**, puis « Connect » à nouveau.

6.2 Après le flash : configurer le Wi-Fi

Le firmware n'embarque **volontairement aucun** identifiant Wi-Fi — après le flash, l'ESP démarre son propre **hotspot de configuration** :

1. Sur le téléphone/PC, rejoindre le réseau Wi-Fi « **Feeder-Relais Setup** » (mot de passe `feeder1234`).
2. Un **portail captif** s'ouvre (sinon accéder à <http://192.168.4.1>).
3. Sélectionner ton Wi-Fi domestique, saisir le mot de passe, enregistrer — l'ESP redémarre et se connecte.
4. Désormais accessible à <http://feeder-relais.local>.

Persistence & réinitialisation d'usine :

- Les données Wi-Fi saisies sont stockées sous une **clé de stockage fixe** et restent **conservées lors des mises à jour OTA/web** (chap. 6.6) — à configurer une fois, puis plus jamais.
- Pour une **réinitialisation d'usine** (effacer les anciennes données Wi-Fi), flasher l'**image factory avec effacement de la flash** : cocher « Erase all flash » dans le flasher web, ou

`esptool.py erase_flash` avant l'écriture. Une simple mise à jour OTA n'efface **pas** les données.

6.3 Résolution de problèmes lors du flash

Symptôme	Cause / Solution
L'ESP n'est pas du tout reconnu	câble de charge au lieu d'un câble de données → autre câble USB-C, autre port USB
Aucun port dans le navigateur	utiliser Chrome/Edge (WebSerial) ; sous Windows éventuellement pilotes CH340/CP210x ; un C3 avec USB natif n'en a généralement pas besoin
« Connect » échoue	mode boot : maintenir BOOT + appuyer RESET + relâcher BOOT
Linux « Permission denied »	utilisateur dans le groupe <code>dialout</code> (<code>sudo usermod -aG dialout \$USER</code>)
Pas de <code>...local</code> après le flash	d'abord configurer le Wi-Fi via le hotspot de configuration (chap. 6.2) ; le mDNS prend un instant

Après le premier flash, toutes les mises à jour se font **sans fil** (chap. 6.6) — la prise USB peut donc rester difficilement accessible une fois l'appareil monté.

6.4 Utilisation web et API JSON (port 80)

`http://feeder-relais.local` ouvre une web-app mobile avec cinq onglets :

- **Accueil** : trois grands boutons (déclenchent T1/T2/T3 avec leur durée réglée), compte à rebours en direct et **Arrêt**.
- **Durées (Réglages)** : tout en haut le **choix de la langue** (Deutsch/English/Français), en dessous les trois durées de minuteur (1–600 s), enregistrées de façon persistante.
- **Réseau (configurable)** : Wi-Fi (SSID/mot de passe), mode IP **DHCP/statique** (+ IP/passerelle/masque/DNS), **serveur NTP**, un **nom d'hôte** (par défaut `feeder-relais`), un interrupteur **roaming Wi-Fi (802.11k/v)** et un bouton **Redémarrage**.
- **État** : tout en haut la **version** flashée, puis la version du firmware, la durée de fonctionnement, la mémoire libre, le Wi-Fi, le SSID avec **canal · barre de signal colorée · signal** (dBm ; 1 barre rouge, 2 jaune, 3–4 vert), **roaming Wi-Fi (802.11k/v) MARCHÉ/ARRÊT**, IP/MAC, cause de reset, relais.
- **Service : log** en direct (niveau d'affichage ERROR/WARN/INFO/DEBUG au choix, activable), **mise à jour du firmware** (téléversement .bin, chap. 6.6) et **redémarrage**.

Langue (interface multilingue) : La web-app et l'OLED parlent **allemand, anglais et français**. Le choix se trouve **tout en haut de l'onglet Durées** et s'applique à **tout l'appareil** (web et OLED), enregistré dans `g_netcfg.lang` (persistant, par défaut allemand). Les textes web sont fournis sous forme de dictionnaire JS dans la page (changement sans rechargement) ; les valeurs d'état dépendant de la langue (état Wi-Fi, cause de reset) proviennent de la firmware sous forme de codes neutres et sont traduites dans le navigateur. L'OLED traduit les jours de la semaine, l'état (Repos/Repas) et les titres de menu (volontairement sans accents en raison du jeu de caractères de l'écran). Le manuel lui-même existe en plus en [anglais](#) et en [français](#) (la version allemande fait foi).

Dans l'**en-tête** figurent le **nom d'hôte** et un **point d'état** : vert = tout va bien, jaune = un minuteur est en cours, rouge = anomalie à l'état de repos (p. ex. OLED injoignable ou pas de Wi-Fi). **Le même code couleur** est affiché sur l'appareil par la **LED RGB onboard** atténuée (WS2812 sur GPIO8, voir chap. 5.2).

Techniquement, `firmware/timer_web.h` (avec `firmware/net_config.h` pour la config réseau persistante) fournit cette page comme un `AsyncWebHandler` propre sur `web_server_base` (l'UI web native d'ESPHome est désactivée). La même **API JSON** est aussi utilisée par l'intégration (p. ex. `ioBroker`) — paramètres en query-string, méthode GET **ou** POST :

Endpoint	Effet
GET <code>/api/status</code>	JSON avec toutes les valeurs : <code>active</code> , <code>remaining</code> , <code>relay</code> , <code>last</code> , <code>times[3]</code> , <code>host</code> , <code>ip</code> , <code>ssid</code> , <code>rsssi</code> , <code>mac</code> , <code>ap</code> , <code>fw</code> , <code>ver</code> , <code>uptime</code> , <code>heap</code> , <code>wifi</code> , <code>reset</code> , <code>lang</code> , <code>roaming</code> (<code>ver</code> = version flashée ; <code>wifi/reset</code> sont des codes neutres du point de vue de la langue, traduits par le JS web)
POST <code>/api/trigger?button=N</code>	déclencher le bouton N (1-3) (utilise sa durée réglée)
POST <code>/api/trigger?seconds=N</code>	commuter ad hoc pendant N secondes
POST <code>/api/stop</code>	couper immédiatement
POST <code>/api/config?time1=A&time2=B&time3=C</code>	régler les durées (chacune 1-600 s, persistant) — champs aussi individuellement
GET <code>/api/net</code>	lire la config réseau : <code>static</code> , <code>ip</code> , <code>gw</code> , <code>sn</code> , <code>dns</code> , <code>ntp</code> , <code>hostname</code> , <code>roaming</code> , <code>lang</code>
POST <code>/api/net?static=0\ 1&ip=&gw=&sn=&dns=&ntp=&host=&roaming=0\ 1&lang=de\ en\ fr</code>	enregistrer la config réseau (champs individuels, persistant)
POST <code>/api/wifi?ssid=&pw=</code>	définir les identifiants Wi-Fi (reconnecte)
POST <code>/api/reconnect</code>	reconnecter proprement le Wi-Fi (applique notamment un roaming modifié immédiatement)
POST <code>/api/reboot</code>	redémarrer l'appareil
GET <code>/api/log?level=N&since=M</code>	tampon circulaire de log en JSON (lignes de niveau $\leq N$, <code>seq > M</code>) ; niveau 1=ERROR...5=DEBUG

Application de la config réseau : Les valeurs sont stockées dans les préférences ESPHome (flash). Le **serveur NTP** prend effet à la prochaine synchronisation (`esp_sntp_setservername`). L'**IP statique** est appliquée lors du `wifi.on_connect` via ESP-IDF-netif et est donc active après un **redémarrage** (DHCP est le défaut). Le **Wi-Fi** utilise le propre `save_wifi_sta()` d'ESPHome ; le premier accès passe toujours par le portail captif. Le **nom d'hôte** est changé à l'exécution : mDNS (`...local`) immédiatement via `mdns_hostname_set()` ; le nom DHCP (dans le routeur) via un **redémarrage du client DHCP** (`esp_netif_dhcpc_stop/start` → nouveau DISCOVER avec l'option 12), donc lui aussi sans reboot. Le nom apparaît également dans l'**en-tête** de la web-app et dans l'onglet du navigateur. Il est réduit à un label DNS valide (a-z, 0-9, « - »).

Roaming Wi-Fi (802.11k/v) : La capacité est compilée en dur (`enable_btm/enable_rrm` dans le YAML → `CONFIG_WPA_11KV_SUPPORT` dans `wpa_supplicant`) ; le **marche/arrêt** est piloté par l'interrupteur web à l'exécution via `g_netcfg.roaming` (persistant). Marche active dans la config STA les bits 802.11v **BTM** (`set_btm`, l'AP/routeur peut réaffecter l'appareil spécifiquement vers l'AP le plus fort) et 802.11k **RRM** (`set_rrm`, listes d'AP voisins) et désactive le propre scan-roaming d'ESPHome (le pilote prend le relais) ; Arrêt fait l'inverse (`set_post_connect_roaming(true)`). Utile uniquement en présence de **plusieurs points d'accès avec le même SSID** (UniFi/Mesh) et s'ils prennent en charge 802.11k/v. Les bits n'entrent dans la config STA qu'à la **prochaine (re)connexion** ; pour cela, la même carte comporte le bouton « **Reconnecter maintenant** » (`POST /api/reconnect` → `wifi.disable + enable`, le Wi-Fi se coupe brièvement), sinon cela prend effet

au prochain redémarrage. Par défaut : **arrêt**. La case reflète **en continu l'état réel de l'appareil** (depuis `/api/status`) et reste donc visiblement cochée après la reconnexion ; l'état actuel figure aussi sous forme de ligne « **Roaming Wi-Fi (802.11k/v) : MARCHE/ARRÊT** » sur la page **État**.

Persistence (important) : Les gestionnaires web s'exécutent dans la **tâche du serveur web**. Les préférences d'ESPHome (file d'attente globale + NVS) ne sont **pas thread-safe** vis-à-vis de la tâche principale — écrire en flash (`save()` / `sync()`) directement depuis le gestionnaire provoquait des **redémarrages sur panique**. C'est pourquoi `netcfg_save()` ne fait que poser un drapeau ; l'écriture durable réelle (`netcfg_flush()` → `global_preferences->sync()`) est effectuée par **l'intervalle de 1 s dans la tâche principale** (la même tâche que la synchronisation des préférences d'ESPHome). Cela vaut pour **tous** les réglages réseau.

Exemple ioBroker (régler la durée du bouton 1 sur 8 s) : `POST http://feeder-relais.local/api/config?time1=8`.

6.5 Utilisation des trois boutons et de l'OLED

Les boutons sont marqués sur le boîtier : **S1 = Down/Manual**, **S2 = SET**, **S3 = UP** (GPIO3/4/5).

État normal :

- **appui court S1 / S2 / S3** → déclenche le minuteur 1 / 2 / 3 avec la durée réglée (active le Shelly jusqu'à l'expiration de la durée).
- **appui long UP (S3) ≥ 1,2 s** → arrête tous les minuteurs et **désactive** le Shelly.
- **appui long SET (S2) ≥ 3 s** → ouvre le **menu d'information**.

Menu d'information (consultation seulement — la configuration passe par la web-app) :

- **S1** avance, **S3** recule ; **appui court SET** ou **10 s sans appui** referme le menu.
- Pages : 1) Wi-Fi (SSID + signal) · 2) adresse IP · 3) heure + NTP · 4) système (firmware + durée de fonctionnement).

Affichage OLED (128 × 32) :

- **En haut** : barres de signal Wi-Fi (d'après le RSSI) à gauche, la grande **heure** (HH:MM) resp. lorsqu'un minuteur est en cours le **compte à rebours** (p. ex. « 10 s ») au centre, état **Repos/Repas** à droite ; dans le menu, à la place, le compteur de pages (p. ex. « 2/4 »).
- **En bas** : à gauche la **date** (p. ex. « Do 16.07.2026 »), à droite la **mémoire libre** (kB).

La base de temps vient du NTP ; jusqu'à la première synchronisation, l'horloge affiche « --:-- ». Fuseau horaire `Europe/Berlin`. Le jour de la semaine, l'état (Repos/Repas) et les titres de menu suivent la **langue réglée** (chap. 6.4).

6.6 Mises à jour du firmware (OTA)

Après le premier flash USB, **deux voies de mise à jour sans fil** sont en place :

- **OTA réseau (ESPHome)** : `esphome run firmware/timer-relais-c3.yaml` met à jour via Wi-Fi (port 3232) — USB n'est plus nécessaire. Configuré via `ota: platform: esphome`.
- **Téléversement web** : dans l'onglet **Service**, téléverser le `firmware.bin` compilé (ou directement `POST /update` sur le port 80). Configuré via `ota: platform: web_server` — passe par `web_server_base`, l'UI web native d'ESPHome n'est **pas** nécessaire pour cela. L'appareil redémarre automatiquement après la mise à jour.

Le `firmware.bin` est produit par `esphome compile ...` et se trouve sous `.esphome/build/feeder-relais/.pioenvs/feeder-relais/firmware.bin`. Le portail captif peut aussi appliquer des mises à jour.

6.7 Log de service / débogage

L'onglet **Service** affiche, lorsque l'« affichage en direct » est activé, les dernières lignes de log (tampon circulaire, 40 lignes) filtrées selon le niveau choisi (ERROR/WARN/INFO/DEBUG). Technique : `logger: on_message:` écrit chaque ligne dans `firmware/log_ring.h` ; l'endpoint web `GET /api/log?level=&since=` les fournit en JSON. Le logger tourne en `level: DEBUG` (pour `VERBOSE`, relever le niveau dans le YAML). Logs complets en plus via `esphome logs ...` (UART/réseau).

7. KiCad — étape par étape (aussi pour débutants)

Testé sur KiCad 8/9/10 ; les noms de menu peuvent varier légèrement.

7.1 Créer le projet et reprendre le schéma

1. Lancer KiCad → **Fichier** → **Nouveau projet** → nom `timer_ersatzplatine`, dossier p. ex. `hardware/kicad/`.
2. KiCad crée un `timer_ersatzplatine.kicad_sch` vide. **Remplacer ce fichier par le nôtre dans l'explorateur** (même nom !) : y copier `hardware/timer_ersatzplatine.kicad_sch`.
3. Ouvrir le projet, lancer l'éditeur de schéma. KiCad convertit éventuellement le fichier au format actuel — c'est normal, il suffit d'enregistrer.
4. Lancer **Outils** → **Vérifier les règles électriques (ERC)**. Les avertissements sur des « broches d'alimentation non connectées » sont connus et sans gravité (nos symboles n'ont volontairement pas de types de broche d'alimentation) ; les neutraliser éventuellement avec des symboles `PWR_FLAG` sur +5V et GND.

7.2 Vérifier/assigner les empreintes

Outils → **Assigner les empreintes**. Sont préaffectées :

Réf	Empreinte	Action
SW1-3	Button_Switch_THT:SW_PUSH_6mm	vérifier (empreinte broches 6,9 × 4,4)
PS1	Converter_ACDC:Converter_ACDC_HiLink_HLK-PMxx	vérifier contre la mesure du TSP-05 !
U2	Package_S0:SOP-4_4.4x2.6mm_P1.27mm	pour la variante DIP (AQY216 DIP) : Package_DIP:DIP-4_W7.62mm
F1	Fuse:Fuse_5x20mm_Horizontal_ReferenceFuseHolder	ou le porte-fusible choisi
RV1	Varistor:RV_Disc_D12mm_W3.9mm_P7.5mm	le S14K275 convient
R1, C1, C2	THT standard	ok
J1-J3	barrette/bornier	J2 en variante comme 4 grandes pastilles à souder
J4	Connector_PinSocket_2.54mm:PinSocket_1x04_P2.54mm_Vertical	barrette femelle !
U1	— (vide)	à créer soi-même , voir 7.3

7.3 Créer l'empreinte de l'ESP32-C3 Super Mini

1. Ouvrir l'éditeur d'empreintes → nouvelle bibliothèque `timer_project.pretty` (dans le dossier du projet, table « spécifique au projet »).
2. Nouvelle empreinte `ESP32-C3_SuperMini` : deux rangées de pastilles 1×8, pas 2,54, entraxe des rangées 15,24 mm (= 6 × 2,54), pastilles ø1,7/perçage 1,0. Contour 18 × 24 mm sur F.Fab, marquer le côté USB.
3. Attribuer les numéros de pastilles selon l'image du brochage (rangée gauche 5V, G, 3.3, 4, 3, 2, 1, 0 / rangée droite 5, 6, 7, 8, 9, 10, 20, 21) et les affecter au symbole U1 dans le schéma (les noms de broche du symbole 5V/G/3V3/3/4/5/6/8/9 doivent pointer vers les bons numéros de pastille).

7.4 Générer la carte à partir du schéma

Éditeur de schéma → **Outils** → **Mettre à jour le PCB à partir du schéma** (« Update PCB from Schematic »). Toutes les empreintes apparaissent en tas à côté de la carte (encore vide).

7.5 Importer la découpe et les références

1. Éditeur de PCB → **Fichier** → **Importer** → **Graphique** → `hardware/platine_original_geometrie.dxf`.
2. L'unité est automatiquement en **mm** grâce à l'en-tête DXF ; import d'abord sur le layer **User.Drawings**, position (0,0), échelle 1.
3. **Contrôle des cotes** : mesurer T1→T2 avec l'outil de mesure — doit donner **25,6 mm**. Si c'est bon, continuer ; sinon vérifier les réglages d'import.
4. Sélectionner la découpe extérieure (**une seule ligne fermée** — les 6 encoches de bossage ouvertes sur le bord font déjà partie de la découpe, plus de cercles séparés) → Propriétés → changer le layer en **Edge.Cuts**. Tout le reste (croix des boutons, zones d'implantation, fenêtre, keepouts, textes) reste sur les layers User ; copier éventuellement en plus les cadres de zones d'implantation avec leur marquage sur **F.Silkscreen** (aide à l'implantation).
5. Placer les 4 trous de fixation comme empreintes : `MountingHole:MountingHole_3.2mm` exactement sur les croix vertes (position via **E** → saisir les coordonnées du chap. 4.3).

7.6 Placer et router

1. **Les boutons d'abord** : SW1-SW3 exactement sur (13,2/12,3), (38,8/12,2), (64,1/12,0) — saisir la position numériquement, ne pas glisser.
2. J4 sur les croix de broches (première broche = GND sur x 15,0 / y 27,84).
3. PS1/TSP sur le rectangle de la zone d'implantation (broches AC vers le bord gauche), U1/ESP sur sa zone (USB vers le bord supérieur), pastilles J2 au bord de la zone d'implantation du Shelly, J1 + F1 + RV1 dans le coin 230 V en bas à gauche, U2/PhotoMOS entre la zone ESP et les pastilles du Shelly, R1 à côté, C1/C2 sur la ligne 5 V près de U1.
4. Créer les **classes de réseau** (Fichier → Réglages de la carte → Classes de réseau) : - HV (réseaux L_IN, L_F, N, SW_SHELLY, O_LAST) : piste ≥ 0,8 mm (chemin d'alimentation de la sortie O du Shelly ≥ 2 mm à 8 A de charge !), distance à l'intérieur de HV ≥ 1,0 mm. - Standard (très basse tension) : 0,25/0,2 mm.
5. **La règle d'or** : entre chaque réseau HV et chaque réseau de très basse tension ≥ **6,0 mm de distance** — le plus simple via une règle personnalisée (Réglages de la carte → Règles personnalisées) : `(version 1) (rule HV_zu_LV (condition "A.NetClass == 'HV' && B.NetClass != 'HV'") (constraint clearance (min 6.0mm)))` Le seul « rapprochement » autorisé est le PhotoMOS lui-même — son boîtier est le point de séparation (broches 1/2 = LV, broches 3/4 = HV).

6. Pas de plans de masse/zones dans la zone HV ; sous le module TSP, aucune piste étrangère.
7. Obtenir un **DRC** (Design Rules Check) sans erreur.

7.7 Données de fabrication et commande

1. **Fichier → Données de fabrication → Gerber** (layers standard) + fichiers de perçage.
2. Vue 3D (Alt+3) comme dernier contrôle visuel.
3. Fabricant (JLCPCB/PCBWay/Aisler) : 2 couches, FR4 1,6 mm, HASL sans plomb ou ENIG, couleur indifférente. La découpe extérieure, y compris les 6 encoches de bossage ouvertes sur le bord, vient automatiquement d'Edge.Cuts.
4. **Avant l'envoi** : impression papier 1:1 de la carte (Fichier → Tracer → PDF, échelle 1:1), découper et essayer dans le boîtier !

8. Montage et mise en service

1. **Implanter les petits composants** (ordre bas → haut) : R1, C2, U2, boutons (précis !), barrette femelle J4, C1, porte-fusible F1, RV1, borniers.
2. **Préparer l'ESP32-C3** : flasher par USB (chap. 6), tester la connexion Wi-Fi, puis souder sur la carte (barrettes mâles ou directement).
3. **Test USB seul** : alimenter l'ESP par USB (pas de secteur !) — les boutons doivent démarrer le compte à rebours, l'OLED enfiché sur J4 doit afficher, GPIO6 doit commuter (multimètre/LED à l'entrée du PhotoMOS).
4. **Souder le TSP-05**, coller le Shelly au VHB sur la zone d'implantation, quatre fils vers J2 (L, N, SW, O), un fil vers la sortie de charge.
5. **Contrôle visuel + contrôle de continuité** : aucun court-circuit entre L/N/PE, distances ≥ 6 mm respectées, aucune projection de soudure.
6. **Premier essai secteur** : boîtier fermé, mettre sous tension via PRCD/DDR. L'OLED affiche l'heure/l'état → configurer le Shelly par bouton/application (Wi-Fi, Input Mode « Switch ») → test des boutons avec charge.
7. Régler les durées à volonté via <http://feeder-relais.local>.

9. Aperçu des fichiers

Fichier	Contenu	Régénérable ?
hardware/platine_template.py	source de vérité pour toute la géométrie ; paramétrique	source
hardware/ platine_original_geometrie.dxf	DXF générée pour KiCad	oui : python3 platine_template.py (nécessite pip install ezdxf)
hardware/platine_vorschau.png	rendu de la DXF	oui (script de rendu voir CLAUDE.md)
hardware/ timer_ersatzplatine.kicad_sch	schéma complet (KiCad 8+)	maintenu à la main
firmware/timer-relais-c3.yaml	configuration ESPHome	maintenu à la main
firmware/timer_web.h	web-app mobile + API JSON (handler C++ sur web_server_base)	maintenu à la main

Fichier	Contenu	Régénérable ?
firmware/net_config.h	config réseau persistante (mode IP, IP statique, NTP, nom d'hôte, roaming 802.11k/v)	maintenu à la main
firmware/log_ring.h	tampon circulaire de log/débugage pour l'onglet Service (/api/log)	maintenu à la main
firmware/build/*.bin	images de flash prêtes (factory + ota), état actuel	généré
firmware/build_images.sh	compile et met à jour firmware/build/	source
docs/DOKUMENTATION.md	ce document (source allemande, faisant foi)	maintenu à la main
docs/DOKUMENTATION.en.md, docs/DOKUMENTATION.fr.md	manuel en anglais/français (traduction, mise à jour à chaque release)	maintenu à la main
docs/PROJEKTPLAN.md	phases, état, checklists (allemand seulement)	maintenu à la main
CLAUDE.md	instructions de travail pour Claude Code	maintenu à la main
docs/DOKUMENTATION.pdf, docs/PROJEKTPLAN.pdf	versions PDF (obligatoire à chaque modification)	oui : python3 tools/md2pdf.py docs/*.md
tools/md2pdf.py	générateur Markdown→PDF (intègre les images)	source
docs/img/*.png	schémas du guide de flash (chap. 6.1)	généré (PIL)

10. Historique des modifications du modèle de géométrie

Version	Modification
v1	Premières coordonnées à partir de photos redressées en perspective ($\pm 1-2$ mm)
v2	Mesure au scanner : 76,7 × 66,1, boutons/trous/découpe précis (scan côté soudure)
v3	Ajustement au boîtier 105,3 × 79,4, 6 découpes de bossage $\varnothing 10$, ancienne découpe supprimée (= B5)
v4	Correction : pas d'encastrement possible (seulement 1,8 mm en dessous) — fenêtre retirée, Shelly sur la zone d'implantation à droite, 4e trou rétabli, keepouts de nervures
v5	TSP-05 planifié à la place du HLK (en bas à gauche), ESP déplacé en haut à gauche
v6	OLED monté sur le cadre : zone de la fenêtre libérée, raccordement J4
v7	Emplacements marqués (textes DXF)
v8	Montage enfiché de l'OLED : J4 construit verticalement sur le centrage de la fenêtre
v9	Positions des bossages mesurées directement (Silvio 13.07. : pas 45, entraxe des rangées 70, ligne supérieure 6 mm au-dessus du bord supérieur, colonne gauche 6 mm à côté du bord gauche → x -6/39/84, y -6/+64) ; 6 découpes de bossage passées de perçages $\varnothing 10$ à des encoches en U ouvertes sur le bord (plus tolérant à l'insertion) ; rayons d'angle réduits localement à 3,2/2,1 aux encoches extérieures ; la découpe reste une boucle Edge.Cuts fermée (vérifié)